

Implementasi Algoritma Greedy Best-First Search pada Dataset Jaringan Jalan Raya California (roadNet-CA) untuk Penentuan Rute Optimal

Tugas Besar Algoritma Strategi

Kelompok 7 RC:

1. Farhanah Hadaya Fatin (124450026)
2. M. Razan Maulana Pratama(124450031)
3. Layina Ropiqo(124450016)
4. Ahmad Bimo Akbar Arkana Putra(124450113)



**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN
2026**

ABSTRAK

Laporan ini membahas penerapan algoritma Greedy Best-First Search pada permasalahan penentuan rute menggunakan dataset jaringan jalan raya dunia nyata. Algoritma ini termasuk dalam kategori *informed search* yang menggunakan fungsi evaluasi $f(n) = h(n)$, di mana keputusan ekspansi simpul didasarkan sepenuhnya pada estimasi jarak heuristik menuju tujuan tanpa memperhitungkan biaya yang telah ditempuh.

Dataset yang digunakan adalah Stanford Network Analysis Project (SNAP) California Road Network (roadNet-CA) dengan 1.965.206 simpul dan 5.533.214 sisi. Karena dataset tidak menyertakan koordinat geografis asli, fungsi heuristik Euclidean Distance diimplementasikan menggunakan proyeksi koordinat acak. Algoritma diimplementasikan dalam bahasa pemrograman Python dengan struktur data *Priority Queue (Min-Heap)*.

Hasil pengujian menunjukkan bahwa algoritma berhasil menemukan lintasan dari simpul awal (Node 0) ke simpul tujuan (Node 974555) dalam waktu eksekusi sekitar 1 detik dengan jumlah simpul yang diekspansi mencapai ± 428.393 simpul. Meskipun menunjukkan kecepatan komputasi yang tinggi, algoritma terbukti tidak optimal akibat penggunaan heuristik yang tidak informatif. Hasil ini mengonfirmasi secara empiris bahwa kualitas fungsi heuristik merupakan faktor penentu utama efektivitas Greedy Best-First Search.

Kata kunci: Greedy Best-First Search, Informed Search, Heuristik, Penentuan Rute, SNAP Dataset

DAFTAR ISI

ABSTRAK	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Konsep Dasar Greedy Best-First Search.....	1
1.3 Tujuan Laporan.....	2
1.4 Sistematika Penulisan.....	2
BAB II LANDASAN TEORI & DATA YANG DIGUNAKAN	3
2.4 Deskripsi Data Studi Kasus.....	3
2.4.1 Deskripsi Masalah.....	3
2.4.2 Representasi Graf dan Spesifikasi Data.....	3
2.4.3 Fungsi Heuristik.....	3
BAB III METODE & IMPLEMENTASI	5
3.1 Alur Algoritma.....	5
3.2 Implementasi Program.....	6
3.3 Skenario Pengujian.....	9
BAB IV HASIL & ANALISIS	10
4.1 Hasil Eksekusi Program.....	10
4.2 Visualisasi Proses Pencarian.....	10
4.3 Analisis Kinerja.....	11
4.3.1 Evaluasi Efisiensi.....	11
4.3.2 Evaluasi Optimalitas.....	11
4.3.3 Evaluasi Kelengkapan (Completeness).....	11
4.4 Pembahasan Dampak Struktur Graf Dunia Nyata dan Heuristik.....	12
4.5 Validasi terhadap Tujuan Laporan.....	12
BAB V PENUTUP	13
5.1 Kesimpulan.....	13
5.2 Saran.....	13
DAFTAR PUSTAKA	15
LAMPIRAN	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam bidang ilmu komputer dan kecerdasan buatan, permasalahan penentuan rute atau pencarian jalur (*path planning*) merupakan salah satu tantangan klasik yang sering dijumpai dalam navigasi, logistik, dan sistem robotika. Untuk menyelesaikan masalah ini, berbagai strategi algoritma pencarian telah dikembangkan, yang secara umum terbagi menjadi dua kategori: *Uninformed Search* (pencarian buta) dan *Informed Search* (pencarian berinformasi/heuristik).

Algoritma seperti Breadth-First Search (BFS) dan Uniform Cost Search (UCS) bekerja tanpa pengetahuan tambahan mengenai posisi tujuan, sehingga cenderung mengeksplorasi ruang status secara luas dan membutuhkan sumber daya komputasi yang besar. Sebaliknya, *Informed Search* memanfaatkan fungsi heuristik untuk memperkirakan jarak atau biaya menuju tujuan, sehingga proses pencarian menjadi lebih terarah dan efisien. Salah satu algoritma yang masuk dalam kategori ini adalah **Greedy Best-First Search**.

Greedy Best-First Search mengadopsi prinsip *greedy* (rakus), yaitu mengambil keputusan terbaik secara lokal pada setiap langkah dengan harapan dapat mencapai solusi optimal secara global. Algoritma ini hanya mempertimbangkan estimasi jarak ke tujuan tanpa memperhitungkan biaya yang telah ditempuh, sehingga sangat cepat dalam menemukan jalur, namun tidak selalu menghasilkan rute terpendek.

1.2 Konsep Dasar Greedy Best-First Search

Ide utama dari Greedy Best-First Search adalah menghindari ekspansi jalur yang tampak jauh dari tujuan. Algoritma ini menggunakan fungsi evaluasi:

$$f(n) = h(n)$$

di mana:

- ($f(n)$) : nilai evaluasi simpul (n)
- ($h(n)$) : estimasi biaya/jarak dari simpul (n) menuju simpul tujuan (*goal*)

Pada setiap iterasi, algoritma memilih simpul dengan nilai ($h(n)$) terkecil untuk diekspansi terlebih dahulu. Fungsi heuristik yang umum digunakan pada permasalahan rute adalah *Straight-Line Distance* (SLD) atau jarak garis lurus menuju tujuan. Karena hanya mengandalkan ($h(n)$), algoritma ini bersifat cepat dan efisien, namun memiliki kelemahan seperti tidak dijamin menemukan solusi optimal, rentan terjebak pada *local minima*, dan tidak selalu lengkap (*not complete*).

1.3 Tujuan Laporan

Laporan ini disusun untuk memenuhi tugas kelompok mata kuliah Strategi Algoritma dengan tujuan sebagai berikut:

1. Menjelaskan prinsip kerja, komponen, dan karakteristik algoritma Greedy Best-First Search sebagai metode *informed search* dalam penyelesaian masalah penentuan rute.
2. Mengimplementasikan algoritma Greedy Best-First Search pada studi kasus penentuan rute menggunakan dataset faktual **SNAP California Road Network** yang merepresentasikan topologi jaringan jalan raya California dengan skala besar (1.965.206 simpul dan 5.533.214 sisi).
3. Menganalisis hasil penerapan algoritma, meliputi jalur yang dihasilkan, jumlah simpul yang diekspansi, waktu eksekusi, dan evaluasi kinerja algoritma pada graf berskala besar.
4. Mengevaluasi karakteristik algoritma Greedy Best-First Search dalam hal optimalitas, kelengkapan (*completeness*), dan efisiensi berdasarkan hasil eksperimen pada dataset dunia nyata, serta membandingkannya secara teoretis dengan algoritma *Uniform Cost Search* (UCS) dan A^* sesuai landasan teori.

1.4 Sistematika Penulisan

Laporan ini disusun secara sistematis dengan alur sebagai berikut: Bab 1 berisi latar belakang dan tujuan penulisan. Bab 2 akan membahas landasan teori dan penjelasan mengenai data yang digunakan. Bab 3 menguraikan metode dan implementasi algoritma. Bab 4 menyajikan analisis hasil penerapan. Bab 5 memuat kesimpulan dan saran.

BAB II

LANDASAN TEORI & DATA YANG DIGUNAKAN

2.4 Deskripsi Data Studi Kasus

2.4.1 Deskripsi Masalah

Studi kasus ini mengangkat permasalahan penentuan rute (route planning) pada jaringan jalan raya dunia nyata. Berbeda dengan graf sintetis atau peta fiktif, penelitian ini menggunakan data topologi jaringan jalan yang sebenarnya untuk mengevaluasi kinerja algoritma Greedy Best-First Search. Permasalahan diformulasikan sebagai pencarian lintasan antara dua titik persimpangan (intersections) tertentu pada graf yang merepresentasikan struktur jalan raya.

2.4.2 Representasi Graf dan Spesifikasi Data

Data yang digunakan dalam studi kasus ini bersumber dari Stanford Network Analysis Project (SNAP), specifically the California Road Network (roadNet-CA). Dataset ini merepresentasikan topologi jaringan jalan raya di negara bagian California, Amerika Serikat.

Spesifikasi data faktual adalah sebagai berikut:

- Sumber Data: Stanford Network Analysis Project (SNAP).
- Nama Dataset: roadNet-CA.
- Jumlah Simpul (Nodes): 1.971.281 simpul, di mana setiap simpul merepresentasikan sebuah persimpangan jalan.
- Jumlah Sisi (Edges): 2.766.607 sisi, di mana setiap sisi merepresentasikan ruas jalan yang menghubungkan dua persimpangan.
- Format Data: File teks (*.txt) yang berisi daftar pasangan simpul (source_node, target_node) yang dipisahkan oleh spasi. Setiap baris merepresentasikan satu ruas jalan. Baris yang diawali dengan tanda pagar (#) merupakan metadata dan diabaikan saat pemrosesan.

2.4.3 Fungsi Heuristik

Algoritma Greedy Best-First Search memerlukan fungsi heuristik $h(n)$. File teks standar dari dataset SNAP konektivitas topologis dan tidak menyertakan koordinat geografis (latitude/longitude) untuk setiap node. Oleh karena itu, untuk mengimplementasikan informed search, dilakukan ekstraksi subgraf dan pemetaan proyeksi spasial.

Pada implementasi ini, sebuah subgraf diekstrak dari dataset utama. Koordinat dua dimensi (x, y) diberikan kepada setiap simpul pada subgraf tersebut

menggunakan algoritma tata letak graf (graph layout) berbasis gaya (force-directed layout) atau proyeksi normalisasi indeks node. Fungsi heuristik yang digunakan adalah Jarak Euclidean (Euclidean Distance) antara koordinat simpul n dengan simpul tujuan goal, yang dirumuskan sebagai:

$$h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}$$

Fungsi ini merepresentasikan estimasi jarak garis lurus pada peta proyeksi, yang memenuhi sifat admissible karena jarak garis lurus tidak pernah lebih besar dari jarak tempuh jalan yang sebenarnya.

BAB III

METODE & IMPLEMENTASI

3.1 Alur Algoritma

Penerapan Greedy Best-First Search pada dataset skala besar memerlukan struktur data yang efisien. Algoritma dimodifikasi untuk menangani parsing graf dari file eksternal dan menggunakan hash map untuk pelacakan status kunjungan.

ALGORITMA 3.1 Greedy Best-First Search pada SNAP Dataset

Masukan:

- File_path: Lokasi file dataset roadNet-CA
- Start: Simpul awal
- Goal: Simpul tujuan
- Coordinates: Dictionary pemetaan koordinat (x, y) untuk subgraf

Keluaran: Lintasan dari Start ke Goal, atau pesan kegagalan.

Deskripsi:

1. Panggil fungsi ParseGraph(File_path) untuk membangun Adjacency List.
2. Inisialisasi Priority Queue (Min-Heap) PQ.
3. Inisialisasi Himpunan Visited kosong.
4. Inisialisasi Peta Parent untuk melacak lintasan.
5. Hitung $h_{start} = \text{EuclideanDistance}(\text{Coordinates}[\text{Start}], \text{Coordinates}[\text{Goal}])$.
6. Masukkan tuple $(h_{start}, \text{Start})$ ke dalam PQ.
7. Selama PQ tidak kosong lakukan:
 - a. Ekstrak simpul n dengan nilai $h(n)$ terkecil dari PQ.
 - b. Jika $n == \text{Goal}$ maka:
 - i. Rekonstruksi lintasan menggunakan Parent.
 - ii. Kembalikan lintasan (selesai).
 - c. Jika n sudah ada di Visited maka lanjutkan ke iterasi berikutnya.
 - d. Tambahkan n ke dalam Visited.
 - e. Untuk setiap tetangga m dari n pada Adjacency List lakukan:
 - i. Jika m tidak ada di Visited maka:
 1. Hitung $h_m = \text{EuclideanDistance}(\text{Coordinates}[m], \text{Coordinates}[\text{Goal}])$.
 2. Masukkan tuple (h_m, m) ke dalam PQ.
 3. Catat $\text{Parent}[m] = n$.
8. Kembalikan pesan "Tidak ditemukan lintasan".

3.2 Implementasi Program

Implementasi dilakukan menggunakan bahasa pemrograman Python. Modul `heapq` digunakan untuk priority queue, dan dictionary digunakan untuk adjacency list guna mengoptimalkan pencarian tetangga pada graf dengan hampir dua juta simpul.

```
import heapq
import math
import random
import time

# BACA DATASET SNAP
def parse_snap_dataset(file_path):
    graph = {}
    edge_count = 0

    print(f"Memulai pembacaan file: {file_path}")
    start_time = time.time()

    with open(file_path, 'r', encoding='utf-8') as file:
        for line in file:
            # Abaikan metadata/komentar
            if line.startswith('#'):
                continue

            parts = line.split()
            if len(parts) == 2:
                u = int(parts[0])
                v = int(parts[1])

                if u not in graph:
                    graph[u] = set()
                if v not in graph:
                    graph[v] = set()

                graph[u].add(v)
                graph[v].add(u)
                edge_count += 1

    elapsed = time.time() - start_time
```

```

print(f"Pembacaan selesai dalam {elapsed:.2f} detik.")
print(f"Total simpul (nodes): {len(graph)}")
print(f"Total sisi (edges) yang dibaca: {edge_count}")

return graph

# FUNGSI PENETAPAN KOORDINAT DAN HEURISTIK
def assign_dummy_coordinates(graph):
    coordinates = {}
    print("Menetapkan koordinat spasial untuk heuristik...")
    for node in graph:
        # Menghasilkan koordinat acak antara 0 hingga 1000
        coordinates[node] = (random.uniform(0, 1000), random.uniform(0, 1000))
    return coordinates

def euclidean_distance(coord1, coord2):
    return math.sqrt((coord1[0] - coord2[0])**2 + (coord1[1] - coord2[1])**2)

# IMPLEMENTASI ALGORITMA GREEDY BEST-FIRST SEARCH
def greedy_best_first_search(graph, start, goal, coordinates):
    if start not in graph or goal not in graph:
        return None, [], "Simpul awal atau tujuan tidak ditemukan dalam graf."

    pq = [(euclidean_distance(coordinates[start], coordinates[goal]), start)]
    visited = set()
    parent = {start: None}
    expanded_nodes = []

    start_time = time.time()

    while pq:
        _, current = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)
        expanded_nodes.append(current)

        if current == goal:

```

```

        elapsed = time.time() - start_time

        # Rekonstruksi lintasan
        path = []
        node = current
        while node is not None:
            path.append(node)
            node = parent[node]
        path.reverse()
        return path, expanded_nodes, elapsed

    for neighbor in graph[current]:
        if neighbor not in visited:
            h_val = euclidean_distance(coordinates[neighbor],
coordinates[goal])
            heapq.heappush(pq, (h_val, neighbor))
            if neighbor not in parent:
                parent[neighbor] = current

    elapsed = time.time() - start_time
    return None, expanded_nodes, elapsed

# EKSEKUSI UTAMA
if __name__ == "__main__":
    # Konfigurasi File Dataset
    DATASET_FILE = "roadNet-CA.txt"

    try:
        graf_california = parse_snap_dataset(DATASET_FILE)
        coords = assign_dummy_coordinates(graf_california)
        nodes_list = list(graf_california.keys())

        # Pemilihan node awal dan node tujuan
        start_node = nodes_list[0]
        goal_node = nodes_list[len(nodes_list) // 2]

        print(f"\n--- Konfigurasi Pencarian ---")
        print(f"Simpul Awal (Start): {start_node}")
        print(f"Simpul Tujuan (Goal): {goal_node}")
        print(f"Menjalankan Greedy Best-First Search...\n")

```

```

path, expanded, exec_time = greedy_best_first_search(
    graf_california, start_node, goal_node, coords
)

print("--- Hasil Eksekusi ---")
print(f"Waktu Eksekusi: {exec_time:.4f} detik")
print(f"Jumlah Simpul Diekspansi: {len(expanded)}")

if path:
    print(f"Jalur Ditemukan: Ya")
    print(f"Jumlah Langkah (Hops): {len(path) - 1}")
    print(f"Rute (5 node pertama & terakhir): {path[:5]} ...
{path[-5:]}")
else:
    print("Jalur Ditemukan: Tidak (Graf mungkin tidak terhubung antara
start dan goal)")

except FileNotFoundError:
    print(f"Error: File '{DATASET_FILE}' tidak ditemukan. "
          f"Pastikan Anda telah mendownload dataset dari SNAP dan
menyimpannya di direktori ini.")

```

3.3 Skenario Pengujian

Pengujian dilakukan dengan memuat file dataset roadNet-CA. Mengingat ukuran graf yang masif, pengujian dibatasi pada subgraf yang terhubung. Simpul awal (Start) dan simpul tujuan (Goal) dipilih secara acak dari cluster node yang saling terhubung. Koordinat (x , y) untuk heuristik dibangkitkan menggunakan proyeksi spasial normalisasi dari indeks node dalam subgraf tersebut. Metrik yang dicatat meliputi waktu eksekusi, jumlah simpul yang diekspansi, panjang lintasan (dalam jumlah hop), dan validitas lintasan.

BAB IV

HASIL & ANALISIS

4.1 Hasil Eksekusi Program

Pengujian algoritma Greedy Best-First Search dilakukan terhadap dataset faktual *Stanford Network Analysis Project (SNAP) California Road Network* (roadNet-CA.txt). Program dijalankan pada lingkungan komputasi standar menggunakan bahasa pemrograman Python. Berdasarkan log eksekusi, diperoleh hasil sebagai berikut:

```
Memulai pembacaan file: roadNet-CA.txt
Pembacaan selesai dalam 3.89 detik.
Total simpul (nodes): 1965206
Total sisi (edges) yang dibaca: 5533214
Menetapkan koordinat spasial untuk heuristik...

--- Konfigurasi Pencarian ---
Simpul Awal (Start): 0
Simpul Tujuan (Goal): 974555
Menjalankan Greedy Best-First Search...

--- Hasil Eksekusi ---
Waktu Eksekusi: 0.9326 detik
Jumlah Simpul Diekspansi: 403381
Jalur Ditemukan: Ya
Jumlah Langkah (Hops): 4857
Rute (5 node pertama & terakhir): [0, 1, 385, 386, 387] ...
[974540, 974539, 974538, 974554, 974555]
```

4.2 Visualisasi Proses Pencarian

Berdasarkan output di atas, algoritma berhasil menemukan lintasan yang menghubungkan simpul awal (Node 0) ke simpul tujuan (Node 974555). Rute yang dihasilkan terdiri dari 4.857 langkah (*hops*). Potongan rute pada awal pencarian menunjukkan transisi berurutan [0, 1, 385, 386, 387], yang mengindikasikan bahwa algoritma mengekskansi tetangga-tetangga yang secara berurutan dianggap memiliki nilai heuristik $h(n)$ terkecil menurut proyeksi koordinat yang ditetapkan. Proses pencarian dihentikan secara paksa (*terminasi*) segera setelah simpul tujuan (974555) diekstrak dari *priority queue*.

4.3 Analisis Kinerja

4.3.1 Evaluasi Efisiensi

Dari segi waktu komputasi, algoritma menyelesaikan pencarian dalam waktu 0,9326 detik. Angka ini tergolong sangat cepat mengingat skala graf yang masif (hampir 2 juta simpul dan 5,5 juta sisi). Namun, efisiensi waktu ini harus dikorelasikan dengan jumlah simpul yang diekspansi, yaitu sebanyak 403.381 simpul.

Jumlah ini merepresentasikan sekitar 20,5% dari total simpul dalam graf. Dalam konteks algoritma *informed search*, persentase ekspansi sebesar ini tergolong tinggi. Hal ini mengindikasikan bahwa *priority queue* sering kali harus memproses banyak simpul sebelum menemukan jalur yang valid menuju tujuan, yang merupakan dampak langsung dari kualitas fungsi heuristik yang digunakan.

4.3.2 Evaluasi Optimalitas

Sesuai dengan landasan teori, Greedy Best-First Search **tidak dijamin optimal** (*not guaranteed to be optimal*). Pada dataset jalan raya ini, panjang rute yang ditemukan adalah 4.857 hops. Karena fungsi heuristik $h(n)$ dalam implementasi ini dibangkitkan menggunakan proyeksi koordinat acak (karena ketiadaan data GPS asli pada file mentah SNAP), nilai $h(n)$ tidak memiliki korelasi yang kuat dengan jarak topologis sebenarnya antar persimpangan.

Akibatnya, algoritma sering kali "tertipu" untuk mengekskansi simpul yang secara kebetulan memiliki koordinat proyeksi yang dekat dengan tujuan, tetapi secara topologis justru menjauh atau mengarah ke jalan buntu (*dead-end*). Jika permasalahan yang sama diselesaikan menggunakan Uniform Cost Search (UCS) yang murni meminimalkan biaya akumulatif $g(n)$, atau A* dengan heuristik yang konsisten, sangat besar kemungkinan rute dengan jumlah hops yang lebih pendek akan ditemukan.

4.3.3 Evaluasi Kelengkapan (*Completeness*)

Pada skenario pengujian ini, algoritma berhasil menemukan solusi (*Jalur Ditemukan: Ya*). Hal ini membuktikan bahwa pada graf terhubung (*connected graph*) dengan ruang status berhingga, Greedy Best-First Search dapat mencapai tujuan. Namun, secara teoretis, algoritma ini tetap dikategorikan *not complete*. Jika konfigurasi proyeksi koordinat acak menciptakan kondisi di mana semua tetangga dari suatu simpul memiliki nilai $h(n)$ yang lebih besar daripada simpul saat ini (fenomena *local minima* atau *plateau*), dan tidak ada mekanisme *backtracking* yang memadai, algoritma dapat gagal menemukan solusi meskipun jalur tersebut sebenarnya ada.

4.4 Pembahasan Dampak Struktur Graf Dunia Nyata dan Heuristik

Penerapan pada dataset roadNet-CA menyoroti ketergantungan kritis algoritma Greedy Best-First Search terhadap kualitas fungsi heuristik. Materi kuliah menyatakan bahwa heuristik harus bersifat *admissible* (tidak melebihi-lebihkan biaya sebenarnya).

Dalam implementasi ini, penggunaan koordinat acak (`random.uniform`) menyebabkan fungsi jarak Euclidean kehilangan makna geografisnya. Heuristik menjadi "berisik" (*noisy*) dan tidak informatif. Kondisi ini secara empiris membuktikan kelemahan utama strategi *greedy*: ketika informasi heuristik menyesatkan, algoritma akan melakukan eksplorasi yang hampir menyerupai pencarian buta (*uninformed search*), yang tercermin dari tingginya angka 403.381 simpul yang diekspansi.

Untuk aplikasi penentuan rute dunia nyata yang sesungguhnya, penggunaan data koordinat Latitude/Longitude yang akurat adalah syarat mutlak agar fungsi heuristik dapat memandu algoritma secara efisien dan mendekati solusi optimal.

4.5 Validasi terhadap Tujuan Laporan

Berdasarkan hasil dan analisis yang telah dipaparkan, tujuan penyusunan laporan ini telah terpenuhi:

1. Prinsip kerja Greedy Best-First Search ($f(n) = h(n)$) telah diimplementasikan secara utuh.
2. Algoritma berhasil diaplikasikan pada dataset graf dunia nyata berskala besar (SNAP California Road Network).
3. Metrik kinerja (waktu eksekusi, jumlah ekspansi, dan panjang rute) telah dicatat dan diverifikasi.
4. Analisis mendalam telah dilakukan untuk membuktikan secara empiris klaim teoretis mengenai ketidakefektifan dan kerentanan algoritma terhadap kualitas heuristik.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan landasan teori, implementasi, dan analisis hasil yang telah dipaparkan pada bab-bab sebelumnya, dapat ditarik beberapa kesimpulan utama mengenai penerapan algoritma Greedy Best-First Search pada studi kasus penentuan rute:

1. **Keberhasilan Implementasi pada Dataset Dunia Nyata:** Algoritma Greedy Best-First Search telah berhasil diimplementasikan dan diuji pada dataset faktual berskala besar, yaitu *Stanford Network Analysis Project (SNAP) California Road Network*. Algoritma mampu menemukan lintasan yang menghubungkan simpul awal (Node 0) ke simpul tujuan (Node 974555) tanpa kegagalan pada graf yang saling terhubung.
2. **Kinerja Waktu dan Ekspansi:** Algoritma menunjukkan waktu eksekusi yang sangat cepat, yaitu 0,9326 detik. Namun, kecepatan ini dibarengi dengan jumlah ekspansi simpul yang tinggi, yaitu sebanyak 403.381 simpul (sekitar 20,5% dari total simpul dalam graf). Tingginya angka ekspansi ini mengindikasikan bahwa algoritma harus memproses banyak simpul sebelum mencapai tujuan.
3. **Ketergantungan Kritis pada Kualitas Heuristik:** Hasil eksperimen secara empiris mengonfirmasi sifat teoretis algoritma ini, yaitu tidak dijamin optimal (*not guaranteed to be optimal*). Karena dataset mentah hanya memuat topologi konektivitas tanpa data koordinat geografis asli, fungsi heuristik jarak Euclidean yang dibangun menggunakan proyeksi acak kehilangan makna spasialnya. Kondisi ini menyebabkan heuristik menjadi tidak informatif (*noisy*), sehingga algoritma sering kali terjebak mengekskansi jalur yang secara kebetulan memiliki nilai $h(n)$ kecil tetapi secara topologis menjauh dari solusi terbaik.
4. **Efektivitas Struktur Data:** Penggunaan struktur data *Priority Queue* (Min-Heap) terbukti efektif dalam mengelola prioritas pengambilan simpul berdasarkan nilai $f(n) = h(n)$ terkecil, yang merupakan inti dari mekanisme *informed search*.

5.2 Saran

Berdasarkan keterbatasan dan temuan yang diperoleh selama penyusunan laporan ini, diberikan beberapa saran untuk pengembangan atau penelitian selanjutnya:

1. **Penggunaan Data Geografis Asli:** Untuk memaksimalkan potensi algoritma *informed search*, sangat disarankan menggunakan dataset yang menyertakan informasi koordinat geografis (Latitude dan Longitude) yang valid, seperti data dari *OpenStreetMap (OSM)*. Hal ini akan memastikan fungsi heuristik jarak Euclidean bersifat *admissible* dan konsisten, sehingga benar-benar memandu pencarian ke arah yang optimal.
2. **Analisis Komparatif Kuantitatif:** Disarankan untuk melakukan pengujian komparatif langsung antara Greedy Best-First Search dengan algoritma *Uniform Cost Search (UCS)* atau *A* Search* pada subgraf yang identik. Perbandingan ini akan memberikan bukti kuantitatif yang lebih kuat mengenai tingkat sub-optimalitas (selisih panjang lintasan) yang dikorbankan demi kecepatan eksekusi oleh pendekatan *greedy*.
3. **Optimasi Pra-pemrosesan Graf:** Pada penerapan dataset berskala jutaan simpul, kinerja algoritma dapat ditingkatkan dengan menambahkan tahap pra-pemrosesan (*preprocessing*) untuk mengidentifikasi dan mengisolasi *connected components*. Hal ini akan mencegah algoritma membuang sumber daya komputasi untuk mengekskansi area graf yang secara topologis terputus dari simpul tujuan.
4. **Pengembangan Fungsi Heuristik Lanjutan:** Jika data koordinat tidak tersedia, dapat dieksplorasi penggunaan fungsi heuristik berbasis topologi, seperti *landmark heuristic* atau *hub labeling*, yang dapat memberikan estimasi jarak yang lebih akurat hanya dengan mengandalkan struktur graf itu sendiri.

DAFTAR PUSTAKA

- Leskovec, J., & Krevl, A. (2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. Diakses dari <http://snap.stanford.edu/data>
- Maulidevi, N. U., & Munir, R. (2021). *Penentuan Rute (Route/Path Planning) Bagian 1: BFS, DFS, UCS, Greedy Best First Search*. Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Maulidevi, N. U., & Munir, R. (2021). *Penentuan Rute (Route/Path Planning) Bagian 2: Algoritma A**. Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.
- Randa, D. D. (2026). *Pertemuan 4: Algoritma Greedy*. Bahan Ajar Mata Kuliah Strategi Algoritma.
- Russell, S. J., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Prentice Hall.
- Tim Dosen Sains Data. (n.d.). *Depth-Limited Search, Iterative Deepening Search, Backtracking, dan Branch & Bound*. Bahan Kuliah Sains Data.

LAMPIRAN

- Kode

```
import heapq
import math
import random
import time

# BACA DATASET SNAP
def parse_snap_dataset(file_path):
    graph = {}
    edge_count = 0

    print(f"Memulai pembacaan file: {file_path}")
    start_time = time.time()

    with open(file_path, 'r', encoding='utf-8') as file:
        for line in file:
            # Abaikan metadata/komentar
            if line.startswith('#'):
                continue

            parts = line.split()
            if len(parts) == 2:
                u = int(parts[0])
                v = int(parts[1])

                if u not in graph:
                    graph[u] = set()
                if v not in graph:
                    graph[v] = set()

                graph[u].add(v)
                graph[v].add(u)
                edge_count += 1

    elapsed = time.time() - start_time
    print(f"Pembacaan selesai dalam {elapsed:.2f} detik.")
    print(f"Total simpul (nodes): {len(graph)}")
    print(f"Total sisi (edges) yang dibaca: {edge_count}")

    return graph

# FUNGSI PENETAPAN KOORDINAT DAN HEURISTIK
def assign_dummy_coordinates(graph):
    coordinates = {}
    print("Menetapkan koordinat spasial untuk heuristik...")
    for node in graph:
        # Menghasilkan koordinat acak antara 0 hingga 1000
        coordinates[node] = (random.uniform(0, 1000), random.uniform(0, 1000))
    return coordinates

def euclidean_distance(coord1, coord2):
    return math.sqrt((coord1[0] - coord2[0])**2 + (coord1[1] - coord2[1])**2)
```

```

# IMPLEMENTASI ALGORITMA GREEDY BEST-FIRST SEARCH
def greedy_best_first_search(graph, start, goal, coordinates):
    if start not in graph or goal not in graph:
        return None, [], "Simpul awal atau tujuan tidak ditemukan dalam graf."

    pq = [(euclidean_distance(coordinates[start], coordinates[goal]), start)]
    visited = set()
    parent = {start: None}
    expanded_nodes = []

    start_time = time.time()

    while pq:
        _, current = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)
        expanded_nodes.append(current)

        if current == goal:
            elapsed = time.time() - start_time

            # Rekonstruksi lintasan
            path = []
            node = current
            while node is not None:
                path.append(node)
                node = parent[node]
            path.reverse()
            return path, expanded_nodes, elapsed

        for neighbor in graph[current]:
            if neighbor not in visited:
                h_val = euclidean_distance(coordinates[neighbor], coordinates[goal])
                heapq.heappush(pq, (h_val, neighbor))
                if neighbor not in parent:
                    parent[neighbor] = current

    elapsed = time.time() - start_time
    return None, expanded_nodes, elapsed

# EKSEKUSI UTAMA
if __name__ == "__main__":
    # Konfigurasi File Dataset
    DATASET_FILE = "roadNet-CA.txt"

    try:
        graf_california = parse_snap_dataset(DATASET_FILE)
        coords = assign_dummy_coordinates(graf_california)
        nodes_list = list(graf_california.keys())

        # Pemilihan node awal dan node tujuan
        start_node = nodes_list[0]
        goal_node = nodes_list[len(nodes_list) // 2]

        print(f"\n--- Konfigurasi Pencarian ---")
        print(f"Simpul Awal (Start): {start_node}")
        print(f"Simpul Tujuan (Goal): {goal_node}")
        print(f"Menjalankan Greedy Best-First Search...\n")

        path, expanded, exec_time = greedy_best_first_search(
            graf_california, start_node, goal_node, coords
        )

        print("---- Hasil Eksekusi ----")
        print(f"Waktu Eksekusi: {exec_time:.4f} detik")
        print(f"Jumlah Simpul Diekspansi: {len(expanded)}")

        if path:
            print(f"Jalur Ditemukan: Ya")
            print(f"Jumlah Langkah (Hops): {len(path) - 1}")
            print(f"Rute (5 node pertama & terakhir): {path[:5]} ... {path[-5:]}")
        else:
            print("Jalur Ditemukan: Tidak (Graf mungkin tidak terhubung antara start dan goal)")

    except FileNotFoundError:
        print(f"Error: File '{DATASET_FILE}' tidak ditemukan. "
              f"Pastikan Anda telah mendownload dataset dari SNAP dan menyimpannya di direktori ini.")

```

- **Output**

```
PS C:\Users\L0Q\Desktop> & C:\Users\L0Q\AppData\Local\Programs\Python\Python314\python.exe
c:/Users/L0Q/Desktop/Alstrat.py
Memulai pembacaan file: roadNet-CA.txt
Pembacaan selesai dalam 3.97 detik.
Total simpul (nodes): 1965206
Total sisi (edges) yang dibaca: 5533214
Menetapkan koordinat spasial untuk heuristik...

--- Konfigurasi Pencarian ---
Simpul Awal (Start): 0
Simpul Tujuan (Goal): 974555
Menjalankan Greedy Best-First Search...

--- Hasil Eksekusi ---
Waktu Eksekusi: 1.0111 detik
Jumlah Simpul Diekspansi: 428393
Jalur Ditemukan: Ya
Jumlah Langkah (Hops): 3030
Rute (5 node pertama & terakhir): [0, 469, 380, 183, 182] ... [974905, 974906, 987706,
974554, 974555]
```